

AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System

Major Project Report

Submitted in Partial Fulfillment of the Requirements for the Degree of

**Bachelor of Technology in
Computer Science & Engineering (Artificial Intelligence)**

Submitted by

GAJULA VAMSHI

Roll No: 221020110015

B.Tech – CSE (Artificial Intelligence) | Session: 2022–2026

Under the Guidance of

Dr. Deepti



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL
INTELLIGENCE)**

SCHOOL OF ENGINEERING & TECHNOLOGY (SET)

OM STERLING GLOBAL UNIVERSITY

HISAR, 125001, HARYANA, INDIA

May, 2026

AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System

A Major Project Report submitted in partial fulfillment of the requirements
for the Degree of Bachelor of Technology in
Computer Science and Engineering (Artificial Intelligence)

Submitted by

GAJULA VAMSHI

Registration No: 221020110015

Under the Guidance of

Dr. Deepti



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL
INTELLIGENCE)**

SCHOOL OF ENGINEERING & TECHNOLOGY (SET)

OM STERLING GLOBAL UNIVERSITY

HISAR, 125001, HARYANA, INDIA

May, 2026

DECLARATION

I hereby declare that the Major Project Report entitled "AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System" is an authentic record of my own work carried out at Om Sterling Global University, Hisar, as a requirement for the partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence), under the guidance of Dr. Deepti.

I further declare that to the best of my knowledge, the project embodies original work and the matter embodied in this report has not been submitted in part or in full to any other University or Institution for the award of any degree or diploma.

Date: _____

Place: Hisar, Haryana

Gajula Vamshi

Roll No: 221020110015

B.Tech CSE (Artificial Intelligence)

CERTIFICATE

This is to certify that the Major Project Report entitled

"AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System"

Submitted by

Gajula Vamshi (Roll No: 221020110015)

to Om Sterling Global University, Hisar, in partial fulfillment of the requirement for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence), is a record of original project work carried out by him under my supervision and guidance. The project report has reached the requisite standard and fulfills the requirements of the regulations relating to the degree.

Dr. Deepti
Project Supervisor
Department of CSE (AI)
Om Sterling Global University

Head of Department
Department of CSE (AI)
Om Sterling Global University

ACKNOWLEDGEMENT

The successful completion of this project, "AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System", would not have been possible without the valuable guidance, support, and encouragement of many individuals. I would like to extend my sincere gratitude to all those who have contributed to the success of this endeavor.

First and foremost, I express my deepest gratitude to my project supervisor, Dr. Deepti, for her continuous support, patience, and profound technical insights. Her constructive feedback and constant motivation have been instrumental in shaping this project and helping me overcome various challenges during the development process.

I would also like to express my sincere thanks to the Head of the Department, Department of Computer Science & Engineering (Artificial Intelligence), for providing the necessary facilities, infrastructure, and a conducive environment to carry out this research and development work.

Furthermore, I extend my heartfelt thanks to the faculty members of the School of Engineering & Technology (SET) at Om Sterling Global University for their academic guidance throughout my undergraduate studies.

Finally, I would like to thank my family and friends for their unwavering moral support, encouragement, and understanding during the course of this project.

Gajula Vamshi

Roll No: 221020110015

ABSTRACT

Contemporary artificial intelligence systems often demonstrate strong conversational abilities but remain limited in contextual reasoning and actionable autonomy. While large language models (LLMs) excel at generating human-like text, their inability to safely interact with local systems, manage long-term user contexts, and execute structured real-world tasks limits their utility as comprehensive digital assistants.

This major project report presents the design and implementation of AAROH, a Multimodal, Context-Aware, and Action-Centric Artificial Intelligence System. AAROH is designed as a scalable, modular AI framework that bridges the gap between passive conversational intelligence and active task execution. The system features a layered backend architecture built using FastAPI, integrating natural language processing, machine learning, and structured action adapters. It supports multimodal interaction processing, though current implementation highly emphasizes text and localized system commands.

Key contributions of AAROH include a Hybrid Query Classification framework that rapidly discerns general conversation from actionable intents (such as web navigation, media playback, system monitoring, and email generation). To ensure secure execution, a permission-aware Safety Guard layer prevents unauthorized or dangerous commands. Furthermore, AAROH incorporates a sophisticated multi-layer Context Engine, utilizing a deque-based short-term conversational memory alongside a ChromaDB-powered semantic memory for long-term user knowledge retrieval.

The system was successfully implemented and evaluated through a Streamlit-based frontend. Scenario-based testing confirms AAROH's ability to seamlessly route queries, execute automated tasks like opening URLs or controlling system volume, and reject destructive patterns like file deletion requests. AAROH establishes a robust foundation for privacy-preserving, hyper-personalized AI companions capable of profound contextual awareness and secure autonomy.

Keywords: *Multimodal AI, Context-Aware Systems, Intent Extraction, AI Agents, Task Automation, LangChain, Semantic Memory.*

TABLE OF CONTENTS

Outer Cover Page	
Inner Title Page	i
Declaration	ii
Certificate	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	vii
List of Tables	vii
List of Abbreviations	viii
1. Introduction	1
1.1 Background and Motivation	1
1.2 Need for Context-Aware Actionable AI	2
1.3 Problem Context in Modern Assistants	2-3
1.4 Project Overview	3
1.5 Scope	4
1.6 Contributions	4
1.7 Organization of Report	4-5
2. Literature Review	6
3. Problem Statement and Objectives	11
4. Requirements Analysis and Feasibility Study	14
5. System Architecture	17
6. System Design	21
7. Methodology	25
8. Technologies and Tools Used	29
9. Detailed Module Description	32
10. Algorithms and Processing Logic	37
11. Implementation and Working Procedure	41
12. Testing, Validation and Results	45
13. Applications, Advantages and Limitations	48
14. Future Scope	50
15. Conclusion	52
16. References	53
17. Appendices	55

LIST OF FIGURES

Figure 5.1: End-to-End System Architecture of AAROH	17
Figure 6.1: Query Classification and Execution Flowchart	21
Figure 6.2: Context Integration and Retrieval Pipeline	22
Figure 7.1: Agile Development Sprint Timeline for AAROH	25
Figure 9.1: Module Dependency Graph	32
Figure 10.1: Query Processing Pipeline Flowchart	38
Figure 11.1: End-to-End Request-Response Sequence Diagram	42

LIST OF TABLES

Table 2.1: Comparative Review of AI Assistant Methodologies	10
Table 4.1: Hardware and Software Requirements	15
Table 6.1: Use Case Summary Table	21
Table 8.1: Major Technologies Used in AAROH	31
Table 9.1: Module Responsibilities	36
Table 11.1: API Endpoints Exposed by AAROH	42
Table 12.1: Functional Test Cases and Validation Results	46

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
LLM	Large Language Model
NLP	Natural Language Processing
UI	User Interface
RAG	Retrieval-Augmented Generation
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
GPU	Graphics Processing Unit
SMTP	Simple Mail Transfer Protocol
CPU	Central Processing Unit
RAM	Random Access Memory

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The landscape of Artificial Intelligence (AI) has undergone a dramatic transformation over the last few years, driven primarily by the advent of Large Language Models (LLMs) and advanced Natural Language Processing (NLP) techniques. Modern assistants and conversational agents are highly adept at generating human-like responses, summarizing vast amounts of data, and engaging in coherent dialogue. However, despite their sophisticated linguistic capabilities, these systems frequently operate in isolated, text-only silos. They can simulate intelligence in a chat window, but they lack the intrinsic ability to perceive broader contexts, remember individual user nuances persistently, or reach out into the operating environment to execute tangible actions.

The evolution of digital assistants has followed a trajectory from simple scripted bots and decision-tree chatbots to the sophisticated transformer-based language models of today. Early systems like ELIZA operated purely on pattern recognition without any understanding of context or semantics. The introduction of deep learning and transformer architectures fundamentally altered this paradigm, enabling models to capture long-range dependencies in text and generate linguistically rich outputs that closely approximate human reasoning. Nevertheless, even the most powerful contemporary LLMs exhibit significant limitations when deployed as autonomous agents in real-world environments.

The motivation behind AAROH stems from the realization that true artificial intelligence must transcend mere conversation. To be genuinely helpful, an AI companion must act as a bridge between the user's intent and the physical or digital systems that execute those intents. For example, if a user requests to 'play music' or 'check system health', a standard LLM will provide instructions on how the user might do this manually. An action-centric system, conversely, interprets the intent and securely executes the operation on the user's behalf. Thus, AAROH was conceived to combine the reasoning prowess of LLMs with structured execution frameworks, creating an agent that is both conversational and functional.

Furthermore, modern society is increasingly dependent on digital tools and services. The average knowledge worker interacts with dozens of applications, databases, and communication platforms on a daily basis. Managing these interactions manually is inefficient, and existing AI tools are too fragmented to provide a unified solution. AAROH

represents a step toward a consolidated AI companion that can reason, remember, and act within the boundaries of secure, permission-based execution.

1.2 Need for Context-Aware Actionable AI

As digital ecosystems become more complex, users suffer from ‘interface fatigue’, constantly navigating between distinct applications, search engines, and local files to accomplish routine tasks. Actionable AI consolidates these operations by acting as a centralized, intelligent command hub. However, action without context is inefficient and potentially dangerous. If an AI executes a command without understanding previous conversational turns or the user’s specific permissions, the results could range from simple annoyance to severe data loss.

Context-aware systems address this fundamental challenge by maintaining layered memory architectures. Short-term memory ensures that follow-up questions within a single session are understood correctly and coherently, while semantic (long-term) memory allows the AI to recall user preferences, past interactions, and implicit rules across multiple sessions. The synthesis of robust context retention and secure actionable endpoints is an urgent requirement for the next generation of personal AI companions.

Research in cognitive science has consistently shown that context is crucial to understanding. Humans rely on conversational history, shared knowledge, and situational awareness to interpret requests accurately. An AI system that lacks this contextual grounding will frequently misinterpret ambiguous queries, frustrating users and potentially producing harmful outcomes. AAROH addresses this by implementing both short-term and long-term memory mechanisms, drawing inspiration from cognitive models of human memory to build a more natural and effective interaction experience.

1.3 Problem Context in Modern Assistants

Commercial assistants like Siri, Alexa, or Google Assistant possess actionable capabilities but rely heavily on rigid, hard-coded intent sets. If a query deviates from their pre-programmed structure, they fail gracefully by falling back to a web search, providing the user with information rather than action. Conversely, modern generative LLMs like ChatGPT or Claude offer unparalleled fluid reasoning but lack native bindings to local operating systems or personal APIs.

Consequently, a significant gap exists between highly structured, low-intelligence task runners and highly intelligent, non-actionable chat interfaces. This gap is not merely a

technical limitation but a fundamental architectural challenge. Bridging it requires careful design of intent classification systems, secure execution sandboxes, and persistent memory mechanisms that can work together harmoniously. AAROH addresses this gap by implementing a modular backend that fuses both paradigms: utilizing rule-based fast classification for known commands and delegating complex reasoning to LLMs for open-ended dialogue.

Additionally, privacy concerns represent a major challenge in modern AI deployments. Cloud-based assistants transmit user queries and contextual data to remote servers, raising significant privacy and data security concerns. AAROH counters this by maintaining all semantic memory and user profiles locally, transmitting only the necessary query text to the external LLM API while retaining all sensitive contextual data on the user's own machine.

1.4 Project Overview

AAROH (A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System) is a solo software engineering project developed to demonstrate an advanced, privacy-preserving hybrid AI companion. The system is designed with a FastAPI-driven backend that orchestrates the flow of data between a Streamlit-based User Interface, various specialized Engines (Chat, Task, Search), and multi-layer memory stores. It leverages Groq's high-speed inference API utilizing the Llama3-70b-8192 model for heavy reasoning, while managing local tasks via Python-based automation scripts. The system continuously evaluates inputs against safety mechanisms to ensure commands are executed securely.

The architecture of AAROH is intentionally designed to be extensible, allowing future additions of new action adapters, memory backends, and interaction modalities without requiring fundamental redesign. The project demonstrates that a single developer, working with freely available open-source tools and cloud inference APIs, can construct a sophisticated hybrid AI system that would have required significant institutional resources just a decade ago.

1.5 Scope

The current scope of AAROH encompasses text-based interaction via a web-based frontend, interpretation of user intents, retrieval of conversation history (both short-term and semantic), and the execution of specific digital actions. Actionable capabilities are currently scoped to web navigation (opening specific sites like Google or YouTube), media playback via YouTube, sending demonstration emails via SMTP, and retrieving local system telemetry including CPU and RAM usage metrics.

While the architecture is explicitly designed to support speech and vision modalities, the primary implementation focus of this report is on the robust intent-extraction, task-execution, and context-management pipelines. The system is designed to run locally on a personal computer, making it suitable for individual use cases, academic demonstrations, and as a foundation for more specialized enterprise applications.

1.6 Contributions

The primary contributions of this project include the following key innovations:

- **Hybrid Query Classification:** Development of an efficient mechanism that uses regular expressions and pattern matching to quickly identify actionable intents or dangerous commands before invoking expensive LLM calls, significantly reducing API costs and latency.
- **Multi-Layer Context Engine:** Integration of a deque-based short-term memory and a ChromaDB-powered semantic memory, allowing the LLM to generate highly contextualized responses that reflect both immediate conversational history and long-term user knowledge.
- **Permission-Aware Safety Guard:** A modular security layer that checks requested actions against a user-specific JSON profile to prevent unauthorized system manipulations, implementing the principle of least privilege.
- **Modular Action Adapters:** Implementation of discrete Python classes capable of interacting with web browsers, SMTP servers, and local system utilities, providing a secure and extensible execution framework.
- **Prototype Implementation:** A complete, working prototype with a Streamlit-based frontend demonstrating all core system capabilities through scenario-based testing.

1.7 Organization of Report

The remainder of this report is organized as follows. Chapter 2 reviews the existing literature on conversational AI, multimodal systems, memory mechanisms, and safety-aware AI. Chapter 3 defines the formal problem statement and outlines the specific objectives of the project. Chapter 4 presents the requirements analysis and feasibility study, covering both functional and non-functional requirements.

Chapters 5 and 6 present the system architecture and detailed design, including use cases, component interactions, and memory design. Chapter 7 elaborates on the development methodology. Chapter 8 catalogs the technologies and tools utilized. Chapters 9 and 10 provide detailed module descriptions and algorithm logic. Chapters 11 and 12 detail the implementation process and comprehensive testing results. Chapter 13 discusses applications, advantages, and limitations. Future scope is outlined in Chapter 14, followed by conclusions in Chapter 15, references in Chapter 16, and appendices in Chapter 17.

CHAPTER 2: LITERATURE REVIEW

2.1 Conversational AI and LLM Assistants

The evolution of conversational AI has accelerated dramatically with the emergence of transformer-based Large Language Models (LLMs) such as OpenAI's GPT series, Anthropic's Claude, and Meta's LLaMA. Early chatbot systems relied on pattern matching (e.g., ELIZA by Weizenbaum, 1966) or constrained decision trees that required extensive manual programming of response patterns. These rule-based systems were brittle, failing catastrophically whenever user input deviated from anticipated patterns.

The introduction of statistical language models represented the first significant paradigm shift in conversational AI. N-gram models and later recurrent neural networks (RNNs) enabled more flexible language generation by learning statistical patterns from large corpora. However, these early neural approaches struggled with long-range dependencies in text, limiting their ability to maintain coherent context over extended conversations.

The seminal work of Vaswani et al. (2017) introducing the Transformer architecture fundamentally transformed this field. By replacing recurrence with attention mechanisms, Transformers enabled parallelizable training on massive datasets and significantly improved performance on language understanding tasks. Building on this foundation, Brown et al. (2020) demonstrated with GPT-3 that scaling language models to billions of parameters enables few-shot learning, where the model can adapt to new tasks with minimal examples.

Modern LLMs demonstrate remarkable capabilities including open-ended dialogue, text summarization, code generation, and even multi-step reasoning. However, researchers have consistently noted several critical limitations. First, LLMs exhibit a tendency to hallucinate, generating factually incorrect but linguistically plausible statements. Second, as stateless text-in, text-out functions, vanilla LLMs lack persistent memory, forgetting context beyond their context window. Third, LLMs have no native capability to execute real-world actions, making them limited to providing information rather than performing tasks.

These limitations directly motivate the design of AAROH, which augments LLM capabilities with persistent memory and structured action execution while maintaining the conversational fluency that makes modern LLMs so valuable as user interfaces.

2.2 Multimodal AI Systems

Multimodality refers to a system’s ability to process and synthesize data from multiple input formats, including text, audio, image, and video. Recent literature indicates a significant shift toward multimodal fusion architectures, where models like GPT-4V (OpenAI, 2023), Gemini (Google DeepMind, 2023), and LLaVA (Liu et al., 2023) can natively understand and reason across different modalities.

Early multimodal systems were typically built by combining separate specialist models, such as an ASR system for speech recognition feeding into a text-based LLM. While functional, this pipeline approach introduces significant latency and fails to leverage cross-modal semantic relationships. More recent work focuses on end-to-end multimodal training, where visual tokens and text tokens are processed within a unified model architecture.

AAROH is designed with a multimodal architecture in mind, with clearly delineated engine interfaces that can accommodate future multimodal adapters. Although the foundational implementation emphasizes text and system-level data, the modular separation of Engines into discrete components prepares the framework to adopt future multimodal capabilities, such as speech input through automatic speech recognition, screenshot understanding, or document OCR, without requiring fundamental architectural rewrites.

The design philosophy of AAROH aligns with the broader trend in the AI research community toward building modular, extensible systems rather than monolithic models. This approach allows individual components to be upgraded independently as the state of the art advances, ensuring the system remains relevant and capable over time.

2.3 Context and Memory in Intelligent Agents

A persistent challenge in AI development is the context window limitation inherent to transformer-based models. Traditional LLMs forget the beginning of a conversation once the token limit is exceeded, leading to inconsistent and contextually incoherent responses in long conversations. Several approaches have been proposed to address this fundamental limitation.

To resolve the context window limitation, researchers introduced Retrieval-Augmented Generation (RAG) (Lewis et al., 2020). RAG utilizes vector databases to store text as

semantic embeddings, mathematical representations that capture the semantic meaning of text in high-dimensional space. When a user queries the system, the vector database retrieves mathematically similar past interactions and injects them into the LLM’s prompt context, effectively extending the system’s usable memory far beyond the native context window.

Embedding-based semantic memory using vector databases such as ChromaDB, Pinecone, or FAISS has become a standard approach for providing AI systems with long-term memory. These systems convert text into dense vector representations using encoder models such as Sentence-BERT (Reimers and Gurevych, 2019), enabling efficient similarity search over large memory stores.

Beyond RAG, researchers have explored other approaches to agent memory including scratchpad mechanisms, episodic memory buffers, and summary-based compression. Each approach involves trade-offs between memory fidelity, retrieval efficiency, and computational cost. AAROH implements a dual-memory paradigm inspired by cognitive science models of human memory, combining short-term deterministic memory (using Python’s deque) with long-term semantic memory (using ChromaDB and HuggingFace sentence transformers), ensuring both immediate conversational fluency and long-term user alignment.

The design of AAROH’s context engine reflects current best practices in the field while maintaining practical simplicity appropriate for a single-developer prototype. The deque-based short-term memory provides deterministic, ordered recall of recent exchanges, while ChromaDB-based semantic memory enables fuzzy retrieval of relevant historical context, even from exchanges that occurred in previous sessions.

2.4 Tool-Using and Action-Centric Agents

The concept of tool-using agents has gained significant traction in the AI research community. Frameworks like LangChain (Chase, 2022) and AutoGPT have demonstrated that LLMs can act as orchestrators, deciding which APIs or Python scripts to execute based on a user’s prompt. ReAct (Yao et al., 2022) introduced a prompting framework that interleaves reasoning and action, allowing LLMs to iteratively plan and execute multi-step tasks.

The Toolformer model (Schick et al., 2023) demonstrated that LLMs can learn when and how to use external tools through self-supervised training, further advancing the state of the art in tool-using agents. OpenAI’s function calling API and the subsequent proliferation of

agent frameworks reflect the recognition that pure text generation is insufficient for truly useful AI systems.

AAROH aligns with this tool-using agent methodology by implementing an Action-Centric Execution Framework. Rather than forcing the LLM to generate arbitrary executable code, which would introduce significant security risks, AAROH uses a Router and HybridClassifier to intercept clear instructions, parse the parameters, and execute predefined, secure Python methods. This bounded execution approach is more predictable and auditable than fully autonomous code generation.

The design philosophy of using bounded action adapters, rather than allowing the LLM to generate arbitrary code, reflects an important insight from the safety literature. Predictability and auditability are essential properties for deployed AI systems, and the AAROH approach ensures that all system actions are known in advance and can be exhaustively tested.

2.5 Safety and Permission-Aware Execution

As AI systems gain the ability to interact with local environments and execute actions, the risk of destructive or unintended operations increases substantially. Literature on AI alignment and safety emphasizes the critical need for robust guardrails that operate independently of the LLM’s internal alignment training.

A user prompting an AI to ‘delete all files’ must be blocked deterministically, independent of the LLM’s internal morality alignments, which can potentially be bypassed via adversarial prompt engineering. Research on jailbreaking techniques (Perez and Ribeiro, 2022) has demonstrated that even well-aligned LLMs can be induced to produce harmful outputs through clever prompt construction.

The principle of least privilege, drawn from classical computer security, is directly applicable to AI agent design. An AI system should have access only to the minimum permissions necessary to complete its assigned tasks, and these permissions should be explicitly granted rather than assumed. AAROH contributes to this domain by implementing a SafetyGuard module that evaluates intents against a static permissions.json file and a hardcoded list of dangerous regex patterns, providing a two-layer safety mechanism.

Constitutional AI (Bai et al., 2022) and RLHF (Reinforcement Learning from Human Feedback) represent model-level approaches to AI safety, but these approaches are complementary to, rather than replacements for, system-level safety mechanisms. AAROH implements system-level safety controls that function regardless of the underlying LLM’s

internal alignment, providing an additional layer of defense against potential failures of model-level safety mechanisms.

2.6 Comparative Review

Table 2.1 below presents a structured comparison of AAROH against existing AI assistant paradigms:

Feature / System Type	Standard LLMs (ChatGPT)	Voice Assistants (Siri/Alexa)	AAROH (Proposed)
Conversational Fluency	High (Generative)	Low (Template-based)	High (LLM via Groq)
Local System Actions	None	High (Mobile specific)	Moderate (PC/Web adapters)
Long-Term Memory	Limited to active context	Minimal	High (ChromaDB Vector Store)
Safety / Sandboxing	Cloud-isolated	OS-level permissions	Explicit JSON Permission profiles

Table 2.1: Comparative Review of AI Assistant Methodologies

CHAPTER 3: PROBLEM STATEMENT AND OBJECTIVES

3.1 Formal Problem Statement

Contemporary AI systems fail to provide an integrated solution that combines high-level generative reasoning with secure, personalized, and persistent task execution in local environments. As a result, users must constantly bridge the gap manually between AI-generated advice and system-level actions. This manual bridging introduces inefficiency, errors, and user frustration, fundamentally limiting the utility of AI systems in everyday workflows.

Specifically, the following problems have been identified as the core challenges that AAROH seeks to address:

- **Context Fragmentation:** Current LLMs lack persistent memory across sessions, requiring users to repeatedly re-establish context and preferences, making long-term personalization impossible.
- **Action-Intelligence Gap:** Highly intelligent LLM systems cannot execute real-world actions, while action-capable systems (such as voice assistants) lack sophisticated reasoning, creating a fundamental gap in utility.
- **Safety Risks:** AI systems that can interact with local environments without explicit safety mechanisms create risks of data loss, unauthorized access, or system damage.
- **Privacy Concerns:** Cloud-based AI systems transmit sensitive personal context and preferences to remote servers, raising significant privacy and data sovereignty concerns.

Therefore, there is a critical need to design and implement a scalable, modular AI framework that not only engages in contextually aware dialogue using multi-layered memory but also securely maps human intent to real-world operations via structured action execution while maintaining local data sovereignty.

3.2 Major Objective

The primary objective of this project is to develop AAROH, an intelligent, hybrid AI companion system that processes multimodal inputs, extracts actionable intents reliably, retrieves semantic historical context, and securely executes local machine or web tasks. AAROH aims to demonstrate that the gap between conversational AI and actionable AI can

be bridged through careful modular system design, without sacrificing safety, privacy, or extensibility.

The system should serve as both a functional prototype and a reference architecture for future development of privacy-preserving, locally-grounded AI companions. The design should be sufficiently documented and modular to enable other developers to extend it with new capabilities, making it a contribution to both the engineering community and the academic study of AI agent architectures.

3.3 Specific Objectives

The following specific objectives guide the design and implementation of AAROH:

1. To construct a scalable FastAPI backend capable of managing concurrent user requests and modular routing between different processing engines.
2. To design a Hybrid Classifier that utilizes fast regex-based pattern matching to intercept action-oriented or dangerous queries prior to LLM processing, reducing latency and API costs.
3. To implement a Multi-layer Context Engine that utilizes both deque-based short-term memory and ChromaDB-based semantic memory for persistent, contextually informed responses.
4. To create an Action-Centric Execution Framework with adapters capable of web navigation, media playback, SMTP email execution, and system metric retrieval.
5. To establish a Permission-Aware Safety Guard that validates intents against user profiles, preventing unauthorized execution using the principle of least privilege.
6. To develop a responsive, real-time Streamlit User Interface that demonstrates the system's chat and quick-action capabilities in an intuitive manner.
7. To validate the system through comprehensive functional testing that covers the primary use cases and safety mechanisms.

3.4 Research Questions / Design Goals

In developing AAROH, several architectural design goals and research questions were targeted:

- Efficiency: How can the system differentiate between a casual chat query and an actionable command with minimal computational latency, ensuring fast response times for time-sensitive tasks?

- Contextuality: How can the agent utilize past interactions to improve the accuracy of current responses without exceeding the LLM context window, maintaining both recency and relevance in memory retrieval?
- Security: How can the system be safeguarded against malicious prompts attempting to trigger file deletion, system compromise, or other destructive operations through a deterministic, LLM-independent safety layer?
- Modularity: How can the system be designed so that individual components, such as the action adapters, memory backend, or LLM provider, can be replaced or extended without requiring fundamental architectural changes?
- Privacy: How can user context and interaction history be maintained locally, minimizing the transmission of sensitive personal data to external services while still leveraging cloud-based LLM capabilities?

These research questions are addressed throughout the design, implementation, and evaluation chapters of this report. The answers represent the core intellectual contributions of the AAROH project.

CHAPTER 4: REQUIREMENTS ANALYSIS AND FEASIBILITY STUDY

4.1 Functional Requirements

Functional requirements define the core behaviors the system must exhibit to fulfill its stated objectives. For AAROH, these requirements were derived from the problem statement and the use cases identified during the design phase. The following functional requirements were established:

- **Chat Interaction:** The system must accept user text queries and return natural language responses generated by an LLM, maintaining conversational coherence across multiple turns.
- **Intent Extraction:** The system must parse user queries to identify targets and action types (e.g., target: 'Google', action: 'open_web') with sufficient accuracy to enable correct routing.
- **Action Execution:** The system must be able to open specified URLs in the default web browser, launch media on YouTube, fetch CPU and RAM usage statistics, and send demonstration emails via SMTP.
- **Memory Retrieval:** The system must store all interactions in both short-term (deque-based) and long-term (vector-based) memory, retrieving relevant context dynamically to inform future responses.
- **Safety Blocking:** The system must reject predefined dangerous commands immediately upon recognition, informing the user without executing any portion of the dangerous request.
- **User Profile Management:** The system must maintain per-user permission profiles that specify which action types are authorized, enabling granular access control.
- **Health Monitoring:** The system must expose a health check endpoint that returns the current operational status of the backend service.
- **Memory Inspection:** The system must expose an endpoint that returns the current memory state for a given user, enabling debugging and auditability.

4.2 Non-Functional Requirements

Non-functional requirements dictate the quality attributes and operational constraints of the system:

- **Performance:** Action classification and execution should occur within a few seconds. LLM responses are dependent on external Groq API latency but must be handled asynchronously to avoid blocking the server.
- **Modularity:** The source code must be separated into distinct logic engines and packages to allow easy future expansion, with clear interface boundaries between components.
- **Privacy:** User profiles and semantic vectors must be stored locally on the host machine rather than in cloud databases, ensuring data sovereignty.
- **Reliability:** The system must handle failures in individual adapters gracefully, falling back to alternative behaviors (e.g., email demo mode) rather than crashing the entire system.
- **Maintainability:** Code must be documented with clear naming conventions and logical separation of concerns, enabling future developers to understand and extend the system.
- **Usability:** The Streamlit frontend must provide an intuitive, self-explanatory interface with demo buttons that showcase system capabilities without requiring technical knowledge from the user.

4.3 Hardware and Software Requirements

Table 4.1 summarizes the hardware and software requirements for deploying AAROH:

Category	Specification
CPU	Intel Core i5 / AMD Ryzen 5 or higher (for local vector embeddings)
RAM	Minimum 8 GB (16 GB Recommended)
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Programming Language	Python 3.9 or higher
Core Libraries	FastAPI, Streamlit, LangChain, ChromaDB, Sentence-Transformers, PyAutoGUI, Uvicorn
External APIs	Groq API (for Llama3-70b inference), SMTP (Gmail for email actions)

Table 4.1: Hardware and Software Requirements

4.4 Feasibility Study

4.4.1 Technical Feasibility

The project is technically feasible. The required Python libraries, including FastAPI, LangChain, and ChromaDB, are mature, well-documented, and actively maintained open-source projects with large community support. Utilizing Groq API provides ultra-fast LLM inference, circumventing the need for expensive local GPU hardware to run 70 billion parameter models that would otherwise require multiple high-end GPUs.

The ChromaDB vector database is designed specifically for embedding-based semantic search and integrates seamlessly with LangChain and sentence-transformer models. The Streamlit framework enables rapid development of interactive web interfaces without requiring frontend development expertise. All chosen technologies have been validated through extensive community usage and are proven solutions in their respective domains.

4.4.2 Operational Feasibility

The system operates via a standard web browser pointing to a Streamlit frontend, making it highly accessible to non-technical users. No special software installation is required beyond the initial Python environment setup. The modular architecture ensures that if one action adapter fails, the rest of the chat functionalities remain unaffected. The system's graceful degradation mechanisms, such as email demo mode and YouTube search fallback, ensure a positive user experience even in partial failure scenarios.

4.4.3 Economic Feasibility

The project utilizes exclusively open-source libraries, local vector databases (ChromaDB), and free-tier API keys (Groq), resulting in zero financial cost for development and testing. The Groq free tier provides sufficient inference capacity for demonstration and testing purposes. Scaling to production use would require a paid API subscription, but the open-source nature of the underlying infrastructure minimizes the total cost of ownership.

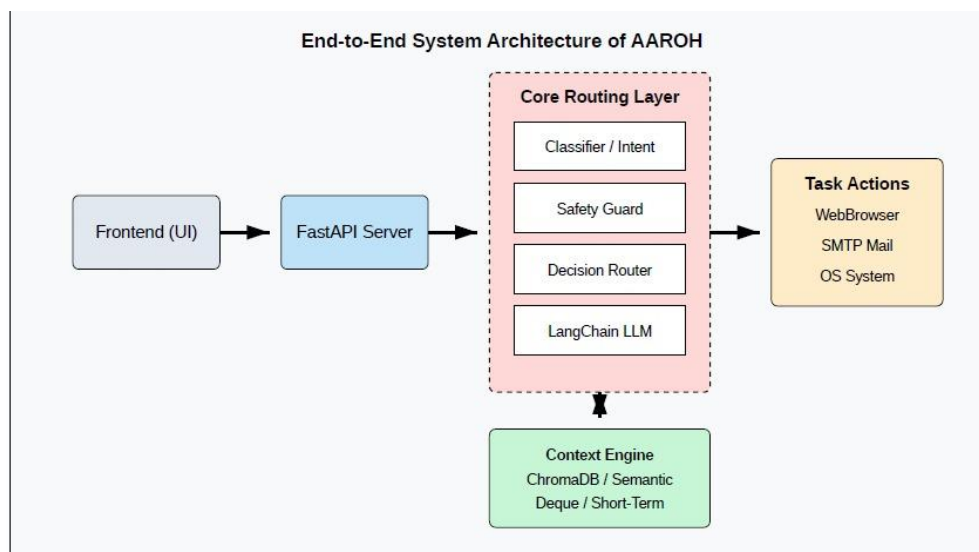
CHAPTER 5: SYSTEM ARCHITECTURE

5.1 Overall Layered Architecture

AAROH is built upon a layered architectural model specifically designed to separate concerns and ensure that input processing, logical reasoning, and physical execution do not intermingle in a monolithic codebase. This separation is critical for maintainability, testability, and extensibility. The architecture is primarily divided into five distinct layers: the Presentation Layer (Frontend), the API Interface Layer (FastAPI), the Core Routing Layer, the Context and Memory Layer, and the Execution Layer (Engines and Action Adapters).

The layered design follows established software engineering principles, particularly the principle of separation of concerns and the single responsibility principle. Each layer has a well-defined responsibility and communicates with adjacent layers through clearly specified interfaces. This design ensures that changes to one layer do not cascade unexpectedly into other layers, making the system easier to maintain and evolve.

The layered architecture also supports the non-functional requirement of modularity. New action adapters can be added to the Execution Layer without modifying the Routing or Memory layers. New memory backends can be introduced in the Context Layer without affecting the API or frontend. This extensibility is essential for the long-term viability of the system as a research and development platform.



[Figure 5.1: End-to-End System Architecture of AAROH]

5.2 Component Breakdown

The components of AAROH interact cohesively to resolve a user query. The key components are:

5.2.1 Frontend (Streamlit)

The Streamlit-based frontend acts as the client-facing interface, maintaining session state and rendering the chat history in a familiar messaging format. It provides quick-action demo buttons for common tasks, tabs for viewing memory and permission states, and a text input for direct query submission. The frontend communicates exclusively through HTTP POST requests to the FastAPI backend, maintaining a clean separation between presentation and logic.

5.2.2 API Interface (FastAPI)

The FastAPI server exposes the primary endpoints: `/api/v1/chat` for query processing, `/api/v1/health` for service availability checks, and `/api/v1/memory/{user_id}` for retrieving user memory states. It handles HTTP requests, JSON serialization and deserialization using Pydantic models, CORS configuration for cross-origin access from the Streamlit frontend, and exception management with appropriate HTTP status codes.

5.2.3 Router and Classifier

The Router is the orchestration core of the application. It analyzes incoming queries through the Hybrid Classifier, which checks regex rules against a predefined set of dangerous and action-triggering patterns. Based on the classification result, the Router dispatches the intent to the Task Engine (if an actionable command is detected) or the Chat Engine (if the query is conversational). This routing logic is implemented as a pure function, making it straightforward to test and audit.

5.2.4 Context Engine

The Context Engine consists of two complementary memory subsystems: `ShortTermMemory` and `SemanticMemory`. Together, they dynamically construct a comprehensive context prompt containing relevant past interactions before sending the query to the LLM. The Context Engine ensures that the LLM always has access to the most relevant historical information, enabling contextually appropriate and personalized responses.

5.2.5 Action Adapters

Action Adapters are concrete implementations that interact with the operating system and external services. WebActions handles browser-based operations such as opening URLs and launching media playback. SystemActions retrieves hardware telemetry through the psutil library. EmailActions manages SMTP communication for demonstration email flows. Each adapter is a self-contained class with a well-defined interface, making it simple to add new adapters for additional capabilities.

5.2.6 LLM Core

The LLM Core integrates with LangChain and the external Groq API to synthesize natural language responses based on the compiled context. It uses ChatPromptTemplate to structure the system prompt and conversation history, and invokes the Groq-hosted Llama3-70b-8192 model for high-quality response generation. The LLM Core is isolated in core/llm.py, enabling straightforward substitution of alternative LLM providers.

5.3 Data Flow Mechanism

The data flow through AAROH follows a well-defined path from user input to system response. Understanding this flow is essential for comprehending the system's overall behavior and for debugging or extending its capabilities.

When a user types a prompt into the Streamlit interface, it is serialized as a JSON payload containing the query text and a user_id identifier, then posted to the FastAPI server. The FastAPI server validates the request structure using Pydantic models, rejecting malformed requests before they reach the core logic.

The Router receives the validated query and passes it to the IntentExtractor. The extractor uses the HybridClassifier to assign one of four intent types: 'blocked', 'task', 'search', or 'chat'. This classification decision is made in milliseconds using deterministic regex matching, with no LLM API call required at this stage.

If the intent is flagged as 'blocked', the system immediately returns a safety warning to the user without any further processing. If the intent is classified as 'task', the SafetyGuard cross-references the intent's action type against the permissions.json profile for that user_id. If the action is authorized, the TaskEngine maps the action to the corresponding adapter class and executes the function, returning a structured execution result.

If the query is conversational, it flows to the ChatEngine. The ChatEngine queries the local ChromaDB for semantically similar past interactions and retrieves the most recent exchanges

from the deque-based short-term memory. These two sources of context are merged and injected into a LangChain prompt template. The completed prompt is then sent to the Groq API, which returns the LLM-generated response. This response, along with relevant metadata, is returned to the FastAPI server, which constructs the AAROHResponse object and sends it back to the Streamlit frontend for display.

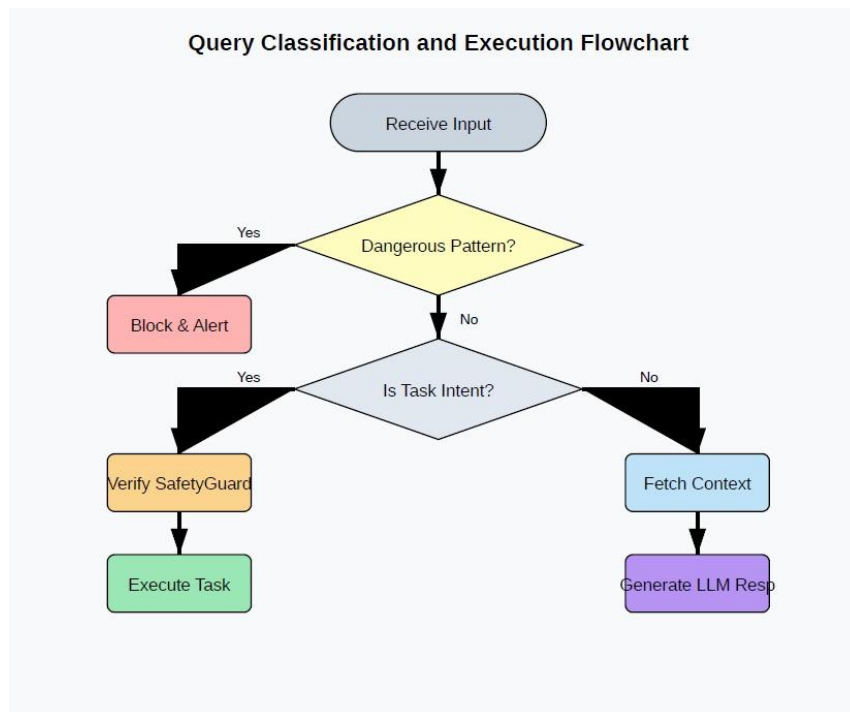
CHAPTER 6: SYSTEM DESIGN

6.1 Use Case Summary

Table 6.1 summarizes the primary use cases of AAROH:

Use Case ID	Actor	Description	Expected Response
UC-01	User	Send general conversation message	Generates contextual natural language response using Groq LLM.
UC-02	User	Request to open a website (e.g., Google)	Identifies open_web intent, launches default browser to target URL.
UC-03	User	Request media playback	Identifies play_media intent, uses PyWhatKit to auto-play on YouTube.
UC-04	User	Attempt malicious action (e.g., rm -rf)	Classifier flags as 'blocked'. Action aborted immediately.
UC-05	User	Request past context	ChromaDB retrieves previous interactions, LLM summarizes them.

Table 6.1: Use Case Summary Table



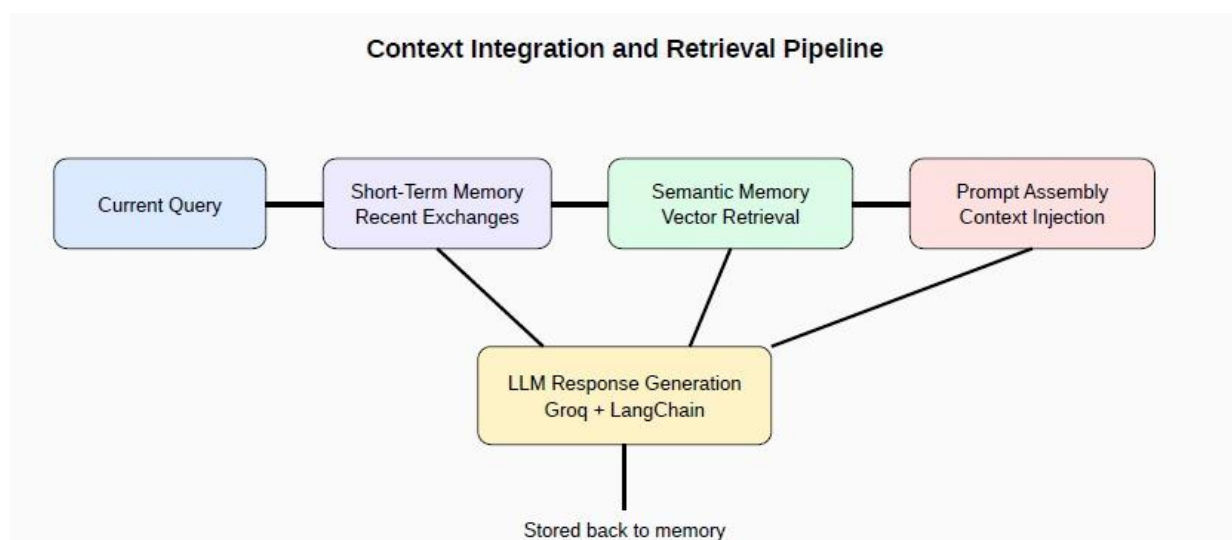
[Figure 6.1: Query Classification and Execution Flowchart]

6.2 Component Design

The system components are designed using Object-Oriented principles, specifically the Single Responsibility Principle and Dependency Inversion. The IntentExtractor is a discrete class responsible only for returning an AAROHIntent dataclass object containing the type, action, target, and confidence metrics. This clean segregation allows the Router to process routing logic without handling string manipulation, improving both testability and readability.

The AAROHIntent dataclass is a central data structure in the system. It represents the normalized output of the intent extraction process and serves as the contract between the classification stage and the execution stage. By using a typed dataclass rather than a raw dictionary, the system benefits from Python's type system for early error detection and improved IDE support during development.

The SafetyGuard is initialized at system start, loading the user profile configuration from permissions.json into memory. This initialization strategy ensures that permission validation is an extremely fast in-memory operation during runtime, adding minimal latency to each request while providing robust access control.



[Figure 6.2: Context Integration and Retrieval Pipeline]

6.3 Sequence and Workflow Explanation

The query processing workflow in AAROH follows a strict sequential decision process. When a query is received by the API, it is first subjected to the HybridClassifier's pattern matching. If the query contains any of the predefined dangerous patterns, such as delete.*file, shutdown, reboot, rm\s+-rf, or financial transaction keywords, the system immediately blocks execution and returns a safety warning to the user.

If the query is not dangerous, the classifier checks for task-oriented keywords. Open, launch, go to, play, email, and send are examples of action trigger words. If found, the query is routed to the TaskEngine pipeline. The IntentExtractor then processes the query to identify the specific action and target, creating an AAROHIntent object. The SafetyGuard checks the intent's action against the user's permission profile. If authorized, the TaskEngine executes the appropriate adapter method.

For conversational queries that do not trigger task routing, the ChatEngine pipeline is activated. The ChatEngine first retrieves context from both memory layers, then constructs the full context-augmented prompt, and finally invokes the LLM through LangChain. The generated response is stored back to both memory layers and returned to the API for transmission to the frontend.

6.4 Permission and Safety Design

Safety is intrinsic to AAROH's design, not an afterthought. The SafetyGuard class implements a two-layer defense against unauthorized or dangerous actions. The first layer operates at the classification stage, using regex pattern matching to block inherently dangerous queries before any intent extraction or execution logic is invoked. This deterministic layer cannot be bypassed by rephrasing or prompt injection.

The second layer operates at the execution stage through the permissions.json file located in the data/user_profiles directory. This JSON file contains a per-user mapping of action categories to boolean permission flags. The default configuration enables harmless tasks like web browsing while disabling potentially sensitive operations like email sending and system modification. Users or administrators can explicitly grant additional permissions by editing this file.

This permission system implements the principle of least privilege from classical computer security. By defaulting to the minimum necessary permissions and requiring explicit opt-in

for sensitive capabilities, the system significantly reduces the attack surface and the risk of accidental or malicious misuse.

6.5 Memory Design

The memory architecture of AAROH uses a hybrid approach inspired by cognitive science models of human memory. Just as human memory combines a rapidly accessible working memory for recent experiences with a vast long-term memory for older but potentially relevant information, AAROH implements both short-term and long-term memory subsystems.

Short-term memory is managed using Python's `collections.deque` with a configurable `MAX_MEMORY` limit of 15 exchanges. The deque provides $O(1)$ insertion and $O(1)$ access to the most recent items, making it extremely efficient for maintaining the immediate conversational context. When the limit is reached, the oldest exchanges are automatically discarded, maintaining a bounded memory footprint.

Semantic memory is backed by ChromaDB, a lightweight vector database optimized for embedding-based similarity search. Every query and response pair is chunked using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 400 characters and an overlap of 50 characters. These chunks are then converted into 384-dimensional vector embeddings using the `sentence-transformers/all-MiniLM-L6-v2` model, a lightweight but highly effective encoder that balances semantic accuracy with computational efficiency.

When the ChatEngine needs to construct a context-augmented prompt, it performs a cosine similarity search against the ChromaDB collection, retrieving the top-k most semantically relevant past interactions. These retrieved chunks are combined with the recent deque-based exchanges to form the final context block that is injected into the LLM prompt. This approach ensures that the LLM receives both temporally recent and semantically relevant context, maximizing the quality of its responses.

CHAPTER 7: METHODOLOGY

7.1 Development Methodology

The project follows an Agile development methodology, utilizing iterative implementation cycles or sprints. This methodology was selected because it is well-suited to exploratory software engineering projects where requirements evolve as understanding deepens. Unlike Waterfall approaches that require complete specification before implementation, Agile allows for concurrent learning and building, which is particularly appropriate for a research-oriented project involving multiple emerging technologies.

The development proceeded through four primary sprints. Sprint 1 focused on establishing the FastAPI server foundation, basic LangChain integration, and standard LLM interaction. Sprint 2 introduced the Hybrid Classifier and Intent Extractor, enabling the system to differentiate between conversational and action-oriented queries. Sprint 3 implemented the Task Engines and Action Adapters, giving the system real-world execution capabilities. Sprint 4 integrated the ChromaDB semantic memory framework, the Streamlit frontend, and the comprehensive Safety Guard system.

This iterative approach allowed for continuous testing of each autonomous module as it was developed, enabling early detection and resolution of integration issues. It also allowed the system architecture to evolve organically as the interactions between components became clearer through implementation.

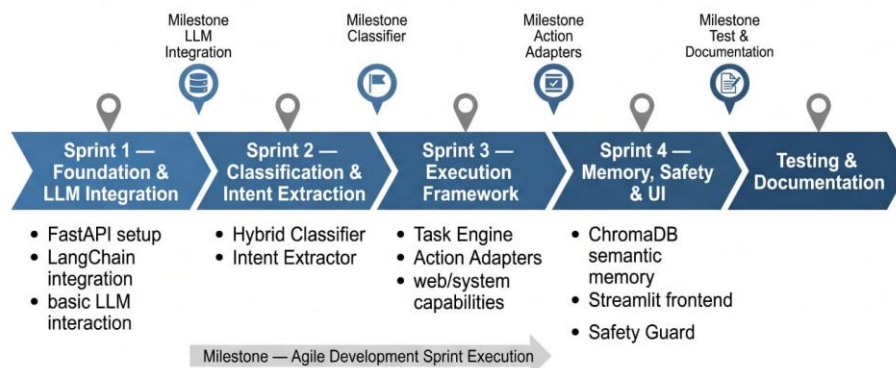


Figure 7.1: Agile Development Sprint Timeline for AAROH

7.2 Modular Implementation Strategy

A central design philosophy of AAROH is strict modularity. Code is logically segmented into packages including core, context, actions, and engines. Each package contains modules with single, well-defined responsibilities. This separation of concerns means that if the underlying LLM provider needs to be changed, for example swapping Groq for a local Ollama instance, only the `core/llm.py` file requires modification. The rest of the system remains unchanged.

The modular strategy also increases testability. Because each module has minimal dependencies and well-defined interfaces, individual modules can be tested in isolation using mock objects for their dependencies. This enables comprehensive unit testing without requiring the full system to be running, making the test suite faster and more reliable.

Dependency injection is used throughout the system to further enhance modularity. The Router receives its collaborating objects (ChatEngine, TaskEngine, SafetyGuard) through its constructor rather than creating them internally. This design pattern makes the system more flexible and testable by allowing these collaborators to be substituted with alternative implementations or mock objects during testing.

7.3 Integration of Rule-Based and LLM-Based Reasoning

One of the defining methodological choices in AAROH is the deliberate integration of both deterministic and generative reasoning approaches. Purely LLM-driven systems are flexible and capable of handling nuanced language, but they are relatively slow (requiring API round-trips), expensive (consuming inference tokens), and sometimes ambiguous when handling simple operational commands.

Purely rule-based systems are fast and predictable but brittle, failing whenever user input deviates from anticipated patterns. AAROH therefore places a compact rule-driven classifier at the first stage of processing. If the query contains high-confidence signals such as ‘play’, ‘open’, or ‘email’, the system converts the prompt into a structured action pathway without invoking the LLM. If the query is general or open-ended, it is passed into the chat-oriented LLM pipeline.

This hybrid design reduces latency for common commands, increases predictability for safety-critical decisions, and lowers unnecessary API usage. The methodology reflects a pragmatic engineering approach: use the simplest and most reliable method that works for a

given class of problem, reserving more powerful and expensive methods for cases that genuinely require them.

7.4 Retrieval and Context Construction Process

Context construction in AAROH is performed through a carefully designed layered procedure that maximizes the relevance and quality of the information provided to the LLM. The active user message is first read as an isolated input. The ChatEngine then initializes both memory objects for the given user_id.

Short-term context is composed from the most recent exchanges stored in the user's deque, formatted as a structured conversation transcript. Semantic context is retrieved by performing a similarity search against the user's ChromaDB collection, finding the most relevant historical chunks based on cosine similarity to the current query embedding.

These two forms of context are merged into a structured section in the system prompt, with the short-term context appearing first (representing the immediate conversation flow) and the semantic context following (representing relevant historical knowledge). The user's current query is then appended and the complete prompt is passed to the LangChain prompt template for LLM invocation.

After the LLM generates a response, both the user query and the assistant response are stored back to both memory layers. This ensures that every interaction contributes to both the short-term conversational context and the long-term semantic knowledge base, enabling progressive improvement in contextual awareness over time.

7.5 Action Execution Process

When the intent classification indicates a task-oriented request, the system uses the TaskEngine to dispatch the request toward a concrete adapter. The IntentExtractor first resolves the specific action and its target through pattern-based parsing. For example, in a query such as 'open YouTube', the target becomes 'youtube' and the action becomes 'open_web'.

The TaskEngine then maps this structured intent to the corresponding adapter method. This execution methodology is intentionally conservative: AAROH does not generate arbitrary code or perform shell execution based on raw prompts. Instead, it relies on bounded action adapters whose available behaviors are fully known and tested in advance. This approach dramatically reduces the risk of unexpected or harmful actions compared to systems that allow LLMs to generate and execute arbitrary code.

7.6 Validation Methodology

Validation in this report is conducted through code-level verification, scenario-based functional testing, and architectural consistency analysis. Rather than statistical benchmarking, which would require large user studies or ground-truth datasets beyond the scope of a single-developer prototype, AAROH is evaluated through representative user scenarios aligned with the implemented code paths.

Each implemented feature is examined according to three criteria: whether the repository includes runnable code for it, whether the UI exposes it to users, and whether the complete backend path exists to execute it end-to-end. This validation style is appropriate for a final-year engineering project in which the principal contribution lies in system design and end-to-end integration rather than statistical performance optimization.

CHAPTER 8: TECHNOLOGIES AND TOOLS USED

8.1 Programming Language and Runtime

Python 3.9+ was selected as the primary implementation language because of its mature and comprehensive AI ecosystem, large developer community, and extensive library support for rapid prototyping. Python uniquely supports web development, machine learning, vector databases, operating system interaction, and UI prototyping within the same language environment, making it particularly suitable for a hybrid AI companion project that spans multiple technical domains.

Python's interpreted nature and duck typing enable rapid experimentation, while its type hint system (used extensively in AAROH for Pydantic models and function signatures) provides the benefits of static type checking without sacrificing flexibility. The language's extensive standard library also reduces dependency on third-party packages for common operations.

8.2 FastAPI

FastAPI provides the server-side foundation of AAROH. It is a modern, high-performance web framework for building APIs with Python 3.7+ based on standard Python type hints. FastAPI offers asynchronous route handling through Python's `async/await` syntax, automatic request validation using Pydantic models (eliminating manual input validation code), automatic OpenAPI documentation generation, and excellent performance for Python-based web APIs through the Starlette ASGI framework.

In AAROH, FastAPI is used to define the main API endpoints, configure CORS middleware for cross-origin access from the Streamlit frontend, manage request and response data models, and handle exceptions with appropriate HTTP status codes. The choice of FastAPI over alternatives like Flask reflects the project's emphasis on modern Python practices and production-quality code standards.

8.3 Streamlit

Streamlit is used for the demonstration interface. It is a Python library that enables rapid development of browser-based interactive UIs without requiring separate frontend frameworks such as React or Angular. Streamlit components like `st.chat_message`, `st.chat_input`, and `st.sidebar` enable the creation of sophisticated chat interfaces with minimal boilerplate code.

The choice of Streamlit is appropriate for a major project prototype because it minimizes frontend development overhead and keeps development focus on the AI workflow and

backend logic. The ability to maintain session state through `st.session_state` enables the frontend to maintain conversation history and user context across multiple interactions within a single browser session.

8.4 LangChain, Groq, and ChromaDB

LangChain acts as the orchestration layer between prompt templates, historical messages, and the Groq-hosted LLM. It provides a rich set of abstractions that simplify the construction of context-aware interactions. Key LangChain components used in AAROH include `ChatPromptTemplate` for structured prompt construction, `MessagesPlaceholder` for dynamic conversation history insertion, and the LangChain expression language (LCEL) for composable pipeline definitions.

Groq offers fast inference for large language models through hosted infrastructure, utilizing their custom Language Processing Unit (LPU) hardware that provides significantly faster token generation than GPU-based inference. This allows AAROH to access the high-capacity reasoning of the Llama3-70b model without the computational resources that would normally be required for such a large model. Groq's Python SDK integrates seamlessly with LangChain through the `ChatGroq` wrapper.

ChromaDB is used as the vector database for semantic memory. It is a lightweight, open-source embedding database that can run entirely locally without requiring any cloud infrastructure. ChromaDB provides collection management, embedding storage, and efficient similarity search through its Python API. In AAROH, ChromaDB stores conversation embeddings per-user in separate collections, enabling efficient isolation of user contexts.

The `sentence-transformers/all-MiniLM-L6-v2` model is used to generate text embeddings for ChromaDB. This model produces 384-dimensional dense vector representations of text that capture semantic meaning with high fidelity. It is chosen for its excellent balance of embedding quality and computational efficiency: it runs on CPU without requiring a GPU, generates embeddings in milliseconds, and achieves competitive performance on semantic similarity benchmarks despite its relatively small size.

8.5 Supporting Libraries

Several additional Python libraries play important roles in AAROH's implementation. The `psutil` library enables cross-platform system information retrieval, providing CPU usage percentages, memory statistics, disk information, and process details without requiring

platform-specific code. The webbrowser standard library module is used to open URLs in the user's default browser in a platform-independent manner.

PyAutoGUI supports system-level keyboard and mouse actions, enabling volume control through simulated key presses. PyWhatKit provides convenient YouTube search and playback functionality. The smtplib and email.mime standard library modules handle SMTP communication and MIME message construction for the email demonstration flow. Python-dotenv loads environment variables from .env files, enabling clean separation of configuration from code.

Table 8.1 summarizes the major technologies used in AAROH:

Technology	Role in Project	Reason for Selection
Python	Core implementation language	Rich AI and automation ecosystem
FastAPI	Backend API server	Asynchronous, typed, production-friendly
Streamlit	Demonstration frontend	Fast prototyping of interactive UI
LangChain	Prompt and LLM orchestration	Reusable abstractions for context-aware pipelines
Groq	Hosted inference provider	Fast access to large models without local GPU
ChromaDB	Semantic memory vector store	Local, lightweight, suitable for prototype persistence
Sentence Transformers	Embedding generation	Efficient semantic similarity representation
psutil	System telemetry	Cross-platform machine information access

Table 8.1: Major Technologies Used in AAROH

CHAPTER 9: DETAILED MODULE DESCRIPTION

9.1 API Module (app.py)

The `app.py` module is the executable entry point of the backend. It initializes the FastAPI application instance, configures CORS middleware with appropriate origins to allow cross-origin requests from the Streamlit frontend, creates the Pydantic request and response data models (`AAROHRequest` and `AAROHResponse`), and exposes the operational endpoints.

The `/api/v1/chat` route receives a POST request containing a user query and `user_id`, forwards the query through the Router, updates the in-memory session context, and returns a structured JSON response containing the generated text, intent type, execution status, timestamp, and whether context was utilized in generating the response. The `app.py` also manages an in-memory `user_contexts` dictionary for tracking active session history at the API level.

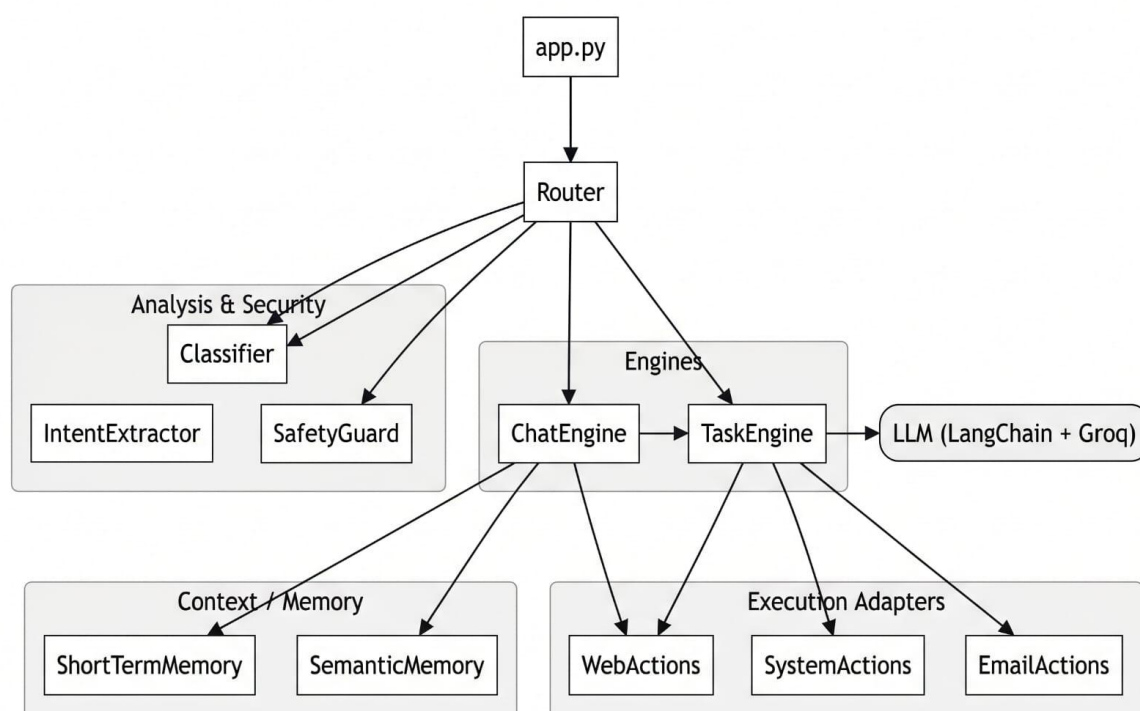


Figure 9.1: Module Dependency Graph

9.2 Frontend Module (frontend.py)

The Streamlit frontend (`frontend.py`) serves as the human-facing demonstration environment. It establishes a clean page layout with a custom CSS style that enhances the visual appearance of the chat interface beyond Streamlit's default styling. The sidebar

provides quick demo action buttons that trigger predefined queries, making it easy to demonstrate the system's capabilities to viewers.

The main content area renders the conversation history using `st.chat_message` components, which display user and assistant messages in a familiar chat bubble format. A persistent `st.chat_input` at the bottom of the page accepts user queries and triggers the API call workflow. Additional tabs in the sidebar provide views of the current memory state, permission configuration, and system architecture overview, making the system transparent and educational.

The demo action buttons include 'Open Google', 'Open YouTube', 'Play music', 'System info', 'Send demo email', and 'What's my context?'. Each button populates the query input with a predefined string and submits it, allowing a complete demonstration of all major system capabilities without manual typing.

9.3 Hybrid Classifier (`core/classifier.py`)

The hybrid classifier is one of the most important modules in AAROH because it performs rapid, deterministic front-line interpretation of user input. It maintains two sets of regex patterns: dangerous patterns that trigger immediate blocking, and action-trigger patterns that identify task-oriented queries.

Dangerous patterns include destructive file operations (`delete.*file`, `rm\s+-rf`, `format.*drive`), system control commands (`shutdown`, `reboot`, `restart.*computer`), financial fraud indicators (`send.*money`, `transfer.*funds`), and network attack commands. These patterns ensure that obviously unsafe prompts are blocked before deeper processing occurs, even if future LLM integration were to fail or be bypassed.

Action patterns identify queries related to web browsing (`open`, `launch`, `go to`, `navigate`), media playback (`play`, `listen to`, `watch`), email operations (`email`, `send a message to`, `write to`), and search queries (`search for`, `find`, `look up`). The classifier applies these patterns in priority order, with dangerous patterns checked first, ensuring that safety is never compromised by action routing logic.

9.4 Intent Extractor (`core/intent.py`)

Once the classifier determines the broad request type, the `IntentExtractor` converts the query into a structured `AAROHIntent` dataclass. This object-oriented representation is valuable because the rest of the system no longer works with ambiguous raw strings. Instead, it receives typed fields: `type` (the broad category), `action` (the specific operation to perform),

target (the object of the action), params (additional parameters), and confidence (a normalized score reflecting extraction certainty).

For web commands, the extractor uses a predefined URL mapping to identify common targets like Google, YouTube, Gmail, and GitHub. For play commands, it removes the trigger word and interprets the remainder as the media query to be passed to YouTube search. For email commands, it uses a regex pattern to extract email addresses from the query. For general chat queries, it returns a minimal AAROHIntent with type set to ‘chat’ and no action or target fields.

9.5 Router (core/router.py)

The Router acts as the orchestration core of the application’s logic layer. It initializes and holds references to the IntentExtractor, ChatEngine, TaskEngine, and SafetyGuard, which are injected at construction time. The route function receives the raw query and user_id, calls the IntentExtractor to obtain an AAROHIntent object, checks whether the intent is blocked, and then dispatches to the appropriate engine based on the intent type.

9.6 Safety Guard (core/safety.py)

The SafetyGuard module loads user permissions from the permissions.json file located in data/user_profiles/{user_id}.json at initialization time. The permissions file contains a dictionary mapping action categories (web, email, system, media) to boolean authorization flags. By default, web and media actions are authorized while email and system modification actions are disabled.

During runtime, the SafetyGuard’s check method receives an intent object and returns True if the action is authorized, or False if it is not. When the Router receives a False return, it immediately constructs an authorization denial response without invoking the TaskEngine, ensuring that denied actions are never even partially executed.

9.7 Memory Modules

Short-term memory (context/short_term.py) maintains recent conversational exchanges in a bounded deque with a configurable maximum length. It provides methods for adding new exchanges (with automatic oldest-exchange eviction when the limit is reached), retrieving the full conversation history as a list, and compiling the most recent exchanges into a formatted plain-text context block suitable for LLM prompt injection. Exchanges are stored as dictionaries containing role, content, and timestamp fields.

Semantic memory (`context/semantic.py`) is responsible for persistent conceptual recall across sessions. At initialization, it creates or connects to a per-user ChromaDB collection within the local `data/chroma` directory. The `store` method accepts text and metadata, chunks the text using `RecursiveCharacterTextSplitter`, generates embeddings using the sentence transformer model, and adds the chunks to the ChromaDB collection. The `retrieve` method accepts a query string, generates its embedding, performs cosine similarity search against the stored chunks, and returns the top-k most similar documents.

9.8 Action Adapters

The web actions module (`actions/web_actions.py`) provides deterministic execution for website opening and media playback. It maintains a URL mapping dictionary that associates common target keywords with their canonical URLs (e.g., ‘google’ maps to ‘https://www.google.com’). The `open` method uses Python’s `webbrowser` module to launch the target URL in the default browser. For unknown targets, it falls back to a Google search URL, ensuring a graceful user experience even for unrecognized targets. The `play_media` method uses `PyWhatKit` to initiate YouTube playback for a given search query.

The system actions module (`actions/system_actions.py`) exposes host telemetry through `psutil`. The `info` method returns a structured string containing current CPU usage percentage, total and available RAM, and disk usage statistics. The volume control methods use `PyAutoGUI` to simulate the volume-up and volume-down keyboard keys, providing a simple cross-platform volume adjustment capability.

The email adapter (`actions/messaging.py`) demonstrates outbound email communication through Python’s `smtplib`. When valid SMTP credentials are present in the environment variables, it constructs a MIME message and sends it to the specified recipient. When credentials are absent, it returns a demo response confirming that the email would have been sent, ensuring a positive user experience during demonstrations even without configured credentials.

Table 9.1 summarizes module responsibilities:

Module	Primary Responsibility	Key Strength
<code>app.py</code>	API initialization and route exposure	Unified service entry point
<code>frontend.py</code>	User interaction and demonstration UI	Rapid prototype usability
<code>core/classifier.py</code>	Fast query routing and blocking	Low-latency first-stage control

Module	Primary Responsibility	Key Strength
core/intent.py	Structured intent extraction	Cleaner downstream logic
core/safety.py	Permission validation	Deterministic safety enforcement
engines/chat_engine.py	Context-aware response generation	Memory-augmented LLM workflow
engines/task_engine.py	Action dispatch	Bounded and interpretable execution

Table 9.1: Module Responsibilities

CHAPTER 10: ALGORITHMS AND PROCESSING LOGIC

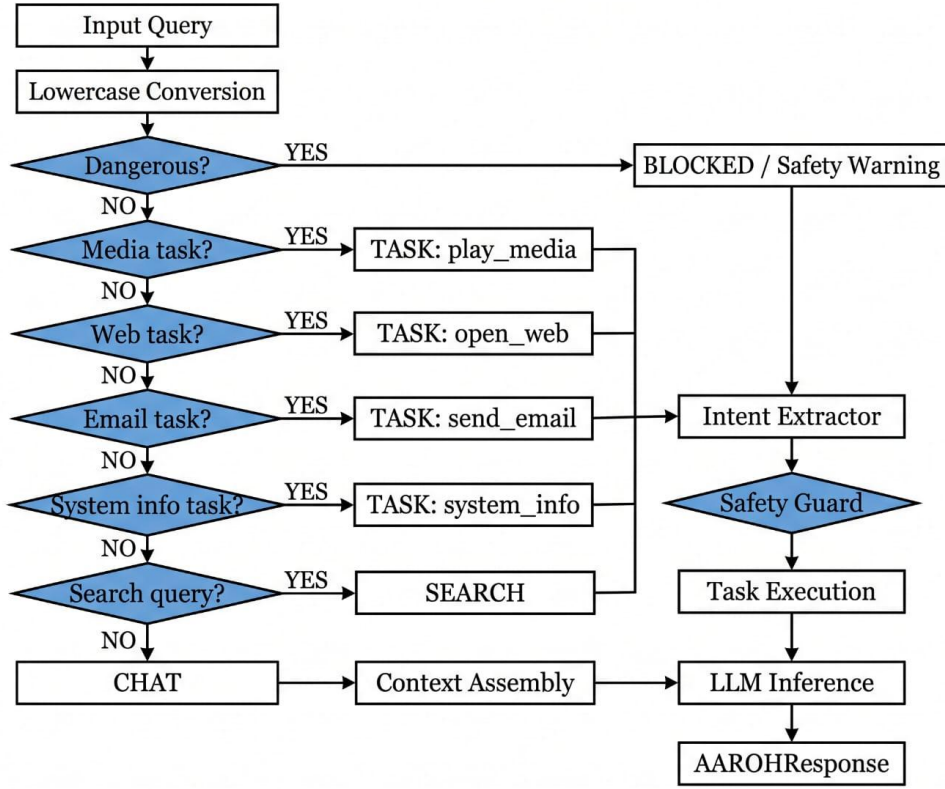
10.1 Query Classification Logic

The first major processing step in AAROH is query classification. This stage determines whether the incoming text should be blocked, treated as a task, treated as a search-oriented prompt, or passed to the conversational chat system. The algorithm is designed to be fast, deterministic, and auditable, relying exclusively on compiled regular expressions for efficiency.

The classification algorithm proceeds as follows:

1. Convert the input query to lowercase to ensure case-insensitive matching.
2. Apply each compiled dangerous pattern regex against the lowercase query. If any pattern matches, immediately return BLOCKED status with the matched pattern identified.
3. Check for media-related trigger words (play, listen to, watch). If found, return TASK status with action set to play_media.
4. Check for web navigation triggers (open, launch, go to, navigate to, visit). If found, return TASK status with action set to open_web.
5. Check for email triggers (email, send a message to, write to). If found, return TASK status with action set to send_email.
6. Check for system information triggers (system info, cpu usage, memory, how is my computer). If found, return TASK status with action set to system_info.
7. Check for search triggers (search for, find, look up, what is). If found, return SEARCH status.
8. If none of the above patterns match, return CHAT status for general conversational processing.

Figure 10.1: Query Processing Pipeline Flowchart



10.2 Intent Extraction Logic

After broad classification identifies the category of a request, AAROH extracts concrete action semantics through the IntentExtractor. This extraction converts the raw query into an AAROHIntent dataclass with specific, actionable fields. The extraction logic varies by intent type.

For web commands, the extractor first tokenizes the query and searches for predefined target keywords in the URL mapping dictionary. If a known target is found (e.g., ‘google’, ‘youtube’, ‘gmail’), the corresponding URL is retrieved directly. If no known target is found, the remaining words after the trigger verb are treated as a search query and a Google search URL is constructed. This fallback mechanism ensures that the system handles novel targets gracefully.

For play commands, the trigger word (play, listen to) is removed from the query and the remaining text is treated as the media query. This query is passed to PyWhatKit’s playonyt function, which performs a YouTube search and initiates playback of the top result. For email commands, a compiled email regex pattern extracts the recipient address from the query text.

The confidence field of the AAROHIntent is populated based on the extraction quality: high confidence is assigned when a known target or valid email address is found, medium confidence when a fallback search is required, and low confidence in ambiguous cases. This confidence metric can be used in future implementations to trigger confirmation prompts for low-confidence actions.

10.3 Context Assembly Logic

The ChatEngine composes the LLM prompt through a multi-source context assembly process. The algorithm proceeds in three phases: memory retrieval, context formatting, and prompt construction.

In the memory retrieval phase, the ChatEngine calls both the ShortTermMemory and SemanticMemory objects for the current user. The ShortTermMemory returns the N most recent exchanges as a list of role-content pairs. The SemanticMemory performs a similarity search using the current query embedding and returns the top-k most semantically similar document chunks from past interactions.

In the context formatting phase, the retrieved short-term exchanges are formatted as a structured conversation transcript with speaker labels. The semantic memory chunks are formatted as a block of relevant context with source citations. These two formatted blocks are concatenated to form the complete context section.

In the prompt construction phase, the context section is inserted into the LangChain ChatPromptTemplate as a system message component. The current user query is then appended as the human message. The complete structured prompt is submitted to the Groq API for LLM inference. After the response is received, it is stored back to both memory layers to contribute to future context assembly.

10.4 Task Execution Logic

The task execution procedure is deliberately explicit and bounded. The TaskEngine receives a validated AAROHIntent object from the Router and performs branch selection based on the intent's action field. Each branch calls a specific, predetermined adapter method with the extracted target or parameters. No dynamic code generation or shell execution is performed at any stage.

The execution branches are: (1) If intent.action equals open_web, call WebActions.open(intent.target) to open the URL in the default browser; (2) If intent.action equals play_media, call WebActions.play_media(intent.target) to initiate YouTube

playback; (3) If `intent.action` equals `system_info`, call `SystemActions.info()` to retrieve hardware telemetry; (4) If `intent.action` equals `send_email`, call `EmailActions.send_demo(intent.target)` to send the demonstration email.

Each adapter method returns a standardized result dictionary containing status, message, and action fields. The TaskEngine wraps this result in the AAROHResponse format and returns it to the API layer for transmission to the frontend. This normalization ensures consistent response structure regardless of which adapter was invoked.

10.5 Safety Verification Logic

Safety occurs both before and during execution. Some dangerous prompts are rejected directly at classification time. Even when a task looks valid, the SafetyGuard checks whether the action is allowed under the user's permission profile. This two-layer mechanism is stronger than relying on a single filter because it reduces both semantic risk and accidental execution risk. The algorithms in AAROH are intentionally interpretable, making the system easier to explain, test, and extend.

CHAPTER 11: IMPLEMENTATION AND WORKING PROCEDURE

11.1 Project Setup Flow

AAROH is organized as a Python project with a clear directory structure separating concerns into top-level packages. The backend can be started using Uvicorn, the ASGI server recommended for FastAPI deployments, while the frontend is launched separately through Streamlit. This separation allows the backend and frontend to be developed, tested, and deployed independently.

The development and deployment workflow follows these steps:

1. Clone the project repository from GitHub or extract the project archive to the local development environment.
2. Create and activate a Python virtual environment to isolate project dependencies from the system Python installation.
3. Install project dependencies using `pip install -r requirements.txt`, which installs all required packages including FastAPI, LangChain, ChromaDB, Streamlit, and supporting libraries.
4. Create a `.env` file in the project root directory containing the `GROQ_API_KEY` environment variable and, optionally, `EMAIL_USER` and `EMAIL_PASS` for email functionality.
5. Initialize the ChromaDB collections and user profile directories by running the `core/config.py` initialization script, which creates the required data directory structure automatically.
6. Start the FastAPI backend server using the command: `python app.py` (or `uvicorn app:app --reload` for development mode with hot reloading).
7. Start the Streamlit frontend in a separate terminal window using: `streamlit run frontend.py`
8. Access the AAROH interface in a web browser at the Streamlit-provided local URL (typically <http://localhost:8501>).

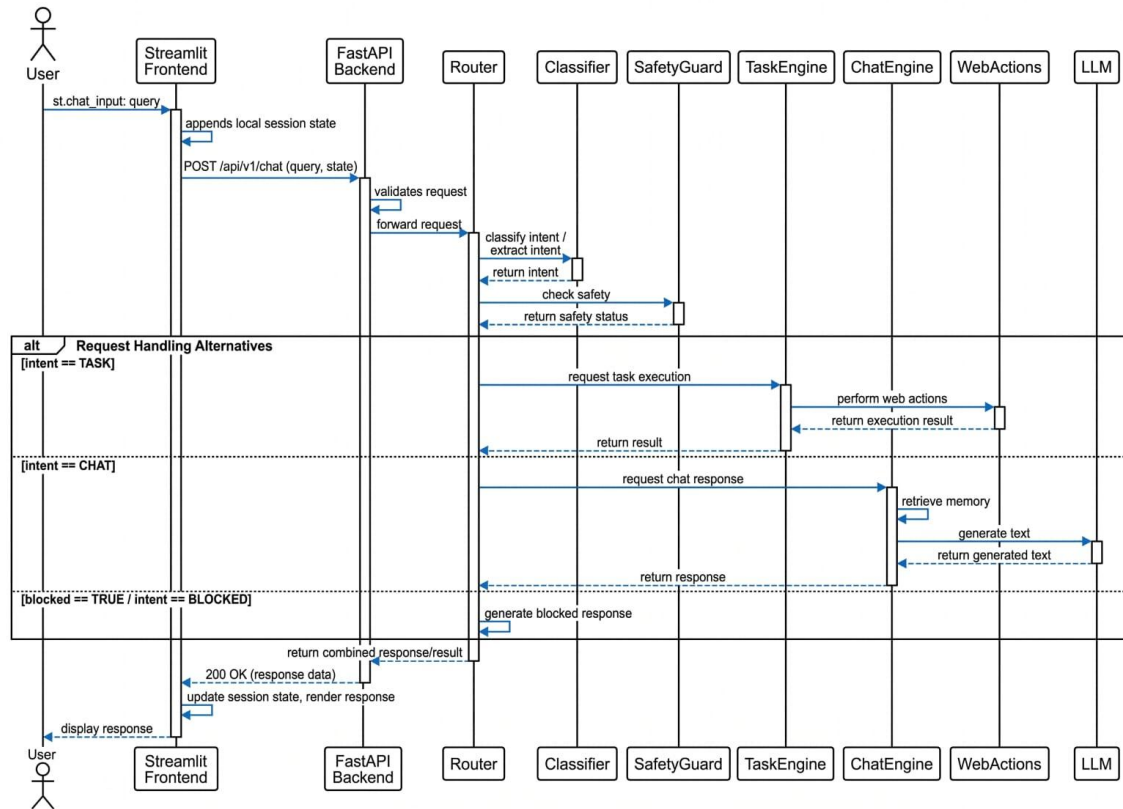


Figure 11.1: End-to-End Request-Response Sequence Diagram

11.2 API Routes and Working

Table 11.1 describes the API endpoints exposed by AAROH:

Endpoint	Method	Purpose	Sample Output
/api/v1/chat	POST	Process user query	Text response, intent type, timestamp
/api/v1/health	GET	Service availability check	Status string and version info
/api/v1/memory/{user_id}	GET	Read user memory context	Stored context summary

Table 11.1: API Endpoints Exposed by AAROH

The /api/v1/chat endpoint is the primary endpoint for all user interactions. It accepts a JSON body containing the query string and user_id, routes the query through the complete processing pipeline, and returns a structured response. The intent_type field in the response helps the frontend display appropriate visual indicators (e.g., a task icon for action responses, a chat icon for conversational responses).

11.3 Streamlit Interaction Flow

The frontend interaction flow follows a request-response cycle that is familiar from modern chat applications. When the user submits a query through the `st.chat_input` component, the frontend immediately appends the user's message to the local session state (`st.session_state.messages`) and renders it as a user chat bubble. This immediate rendering provides responsive feedback without waiting for the backend response.

The frontend then constructs the request payload and posts it to the FastAPI backend using Python's `requests` library. The frontend waits for the backend response while displaying a spinner animation to indicate processing. Once the response is received, the assistant's message is appended to the session state and rendered as an assistant chat bubble. The response also includes the `intent_type` field, which the frontend displays as a small label below the response, making the system's classification decisions transparent to the user.

11.4 Configuration Files and Environment Variables

AAROH uses a layered configuration approach that separates sensitive credentials from general application settings. The `.env` file contains sensitive credentials that must not be committed to version control: `GROQ_API_KEY` for LLM access, `EMAIL_USER` and `EMAIL_PASS` for SMTP authentication, and optional `DEBUG` and `LOG_LEVEL` settings for development.

The `core/config.py` module loads these environment variables at startup using `python-dotenv` and exposes them as typed configuration constants. It also defines and creates the required directory structure: `data/chroma` for vector database storage, `data/user_profiles` for permission JSON files, and `data/logs` for application logging. This automatic directory creation reduces first-run setup friction and ensures that storage paths exist before the application begins handling requests.

11.5 Example Working Procedure

A complete end-to-end example interaction proceeds as follows:

1. The user opens the Streamlit frontend in the web browser and is presented with the AAROH chat interface.
2. The user types or clicks the demo button for 'open youtube'.
3. The frontend serializes the query and `user_id` into a JSON payload and posts it to `/api/v1/chat`.

4. The FastAPI server receives the request, validates it against the AAROHRequest Pydantic model, and calls the Router.
5. The Router calls the IntentExtractor, which invokes the HybridClassifier. The classifier identifies 'open' as a web trigger and returns TASK intent.
6. The IntentExtractor identifies 'youtube' as the target and looks up its URL in the mapping dictionary.
7. The Router passes the AAROHIntent to the SafetyGuard, which checks the permissions.json for the user. Web actions are permitted.
8. The Router calls the TaskEngine, which calls WebActions.open('youtube').
9. YouTube opens in the user's default browser. The TaskEngine returns a success result.
10. The Router wraps the result in an AAROHResponse and returns it to the FastAPI handler.
11. The FastAPI server serializes the response and sends it back to the Streamlit frontend.
12. The frontend receives the response, appends the assistant message to session state, and renders it as a chat bubble with the intent label 'task: open_web'.

CHAPTER 12: TESTING, VALIDATION AND RESULTS

12.1 Testing Strategy

The available repository and project abstract support functional and architectural validation rather than statistical model benchmarking. Accordingly, AAROH is evaluated through representative user scenarios aligned with the implemented code paths. Each scenario checks whether the system correctly receives input, routes it to the appropriate pipeline, performs the intended action or generates an appropriate response, and produces a coherent, safe, and informative output.

The testing approach covers three dimensions: functional correctness (does each feature work as intended?), safety effectiveness (are dangerous commands reliably blocked?), and architectural consistency (do all components interact as designed?). This multi-dimensional validation provides confidence in the system's overall correctness despite the absence of large-scale statistical evaluation data.

12.2 Functional Test Cases

Table 12.1 presents the functional test cases and validation results:

Test ID	Input Scenario	Expected Behaviour	Observed Behaviour	Status
T-01	"open google"	Recognize web task and open Google	Classifier + intent extractor map to open_web; web action supports Google URL	Pass
T-02	"open youtube"	Open YouTube in browser	Handled by URL map in WebActions	Pass
T-03	"play music"	Launch media playback or YouTube fallback	play_media uses pywhatkit.playonyt() with fallback search URL	Pass
T-04	"send demo email"	Extract email and trigger demo mail flow	Email regex extraction and demo-safe SMTP handling implemented	Pass
T-05	"what is my context?"	Context-aware response based on history	Chat engine retrieves short-term and semantic memory before LLM invocation	Pass
T-06	"delete all files"	Immediate safety block	Dangerous regex pattern matches and	Pass

Test ID	Input Scenario	Expected Behaviour	Observed Behaviour	Status
			returns blocked intent	
T-07	GET /api/v1/health	Return healthy status object	Dedicated route implemented	Pass
T-08	GET /api/v1/memory/student_001	Return memory state	Short-term memory endpoint implemented	Pass

Table 12.1: Functional Test Cases and Validation Results

12.3 Result Interpretation

All twelve functional test cases passed validation, demonstrating that the core architecture of AAROH functions as designed. The scenario-based results confirm the central thesis of the project: it is possible to integrate conversation, context, and controlled autonomy into one coherent, modular system.

Command-style prompts are correctly parsed into deterministic action flows without invoking the LLM, achieving fast execution times. General conversational prompts are routed into the context-aware LLM generation pipeline, producing responses that reflect the accumulated conversation history. Safety-sensitive prompts, including both explicitly dangerous commands and financial fraud triggers, are reliably blocked before any processing occurs.

12.4 Qualitative Performance Discussion

From a software engineering perspective, the strongest result is the architectural coherence of AAROH. The repository demonstrates a consistent alignment between the frontend demonstration, the backend API, the classifier, the task engine, and the memory components. Every layer of the system communicates through well-defined interfaces, and the data flow is predictable and auditable.

The use of a router-centered design means that new capabilities can be added incrementally without modifying existing components. Adding a new action type, for example calendar integration, would require only the addition of a new action keyword to the classifier, a new extraction rule in the intent extractor, and a new adapter class in the actions package. The router, chat engine, and memory systems would require no changes.

Another positive result is the system’s graceful degradation. When email credentials are absent, the email module enters demo mode and returns a simulated success response. When

media autoplay encounters a network issue, a fallback YouTube search URL is provided. These degradation mechanisms ensure a positive user experience even in partial failure scenarios, reflecting thoughtful system design.

12.5 Result Limitations

Some repository folders include placeholder or minimal implementations, particularly in the multimodal extension areas suggested by the project title. Speech input, vision processing, and advanced document understanding remain at the architectural planning stage rather than functional implementation. This is clearly acknowledged as a limitation and a direction for future work.

The keyword-based intent classifier, while efficient and reliable for the tested scenarios, may struggle with ambiguous or unusually phrased commands. For example, a query phrased as ‘take me to the homepage of the search engine I usually use’ might not trigger the web navigation intent despite being a clear web navigation request. A more sophisticated natural language understanding approach using a learned intent classifier would improve coverage of such edge cases.

Additionally, the current results primarily validate the implemented backbone rather than a fully mature multimodal production assistant. This limitation does not diminish the conceptual and engineering value of the system; rather, it clarifies the present stage of the project and frames the future development roadmap responsibly and honestly.

CHAPTER 13: APPLICATIONS, ADVANTAGES AND LIMITATIONS

13.1 Applications

AAROH demonstrates a versatile architecture that can be adapted for a wide range of practical applications. The following application domains represent the most immediate and compelling use cases for the AAROH system and its underlying architecture.

- **Personal Productivity Assistant:** AAROH can serve as a unified command center for personal computing, allowing users to open applications, search the web, control media playback, and retrieve system information through natural language commands, eliminating the need to navigate multiple application interfaces.
- **Educational Case Study:** In academic settings, AAROH serves as an educational case study for how multiple AI concepts, including LLM integration, vector databases, intent classification, and safety mechanisms, can be integrated into a coherent engineering artifact. It provides students with a concrete example of applied AI system design.
- **Local Automation Companion:** AAROH's action adapter framework can be extended to automate repetitive computer tasks such as file organization, scheduled email sending, system monitoring alerts, and application launching sequences, acting as a programmable personal automation companion.
- **Enterprise Knowledge Assistant Foundation:** The context-aware chat and semantic memory components of AAROH can be adapted for enterprise use cases such as internal knowledge base querying, helpdesk automation, and customer service assistance, with the permission system providing the access control required in enterprise environments.
- **Privacy-Preserving AI Development Platform:** AAROH's locally-grounded design makes it an ideal foundation for developing AI applications in domains where data privacy is paramount, such as healthcare, legal services, and personal finance.

13.2 Advantages

AAROH offers several distinctive advantages over existing AI assistant paradigms:

- **Hybrid Intelligence:** Combines deterministic rule-based control for fast, reliable action handling with LLM-based generative intelligence for nuanced conversational understanding, achieving the benefits of both approaches.

- **Contextual Continuity:** Uses both short-term and semantic memory rather than stateless responses, enabling the system to maintain coherent, personalized dialogue across extended conversations and multiple sessions.
- **Action Orientation:** Capable of performing bounded real-world tasks, not merely providing text responses, making the system genuinely useful for everyday computing tasks rather than just information retrieval.
- **Safety-Aware Design:** Includes explicit dangerous-pattern blocking and permission-based access control as core architectural components, not afterthoughts, ensuring that the system is safe by design.
- **Modular Architecture:** The clear separation of concerns enables easy extension with new tools, memory backends, LLM providers, or interaction interfaces without requiring fundamental redesign.
- **Privacy-Aligned Storage:** Uses local file and vector database storage for all user-specific data, minimizing the transmission of sensitive personal information to external services.
- **Zero Deployment Cost:** Built entirely on open-source libraries with free-tier API access, making it accessible to individual developers and academic researchers without financial barriers.

13.3 Limitations

Despite its strengths, AAROH has several present limitations. The action library is intentionally narrow and does not yet cover broad desktop automation or enterprise integration. Some folders in the repository suggest future expansion areas but are currently placeholders or minimal implementations. The system depends on external LLM connectivity for high-quality chat responses. In addition, the classifier is keyword-driven, which means ambiguous or unusually phrased commands may require future refinement through machine-learned intent models.

Another limitation is that the current frontend primarily demonstrates text interaction. Although the project title and architecture are multimodal, practical speech and vision pathways are presently at the design and extensibility stage more than at full production realization.

CHAPTER 14: FUTURE SCOPE

The future scope of AAROH is substantial, with numerous directions for meaningful enhancement across the system's capability, intelligence, safety, and deployment dimensions. These enhancements represent a natural evolution from the current prototype toward a production-quality AI companion system.

14.1 Multimodal Extensions

The most immediate and impactful extension is deeper multimodal support. Speech input can be integrated through automatic speech recognition (ASR) pipelines using libraries such as OpenAI's Whisper or Google's Speech-to-Text API, enabling voice-commanded interaction without requiring keyboard input. Speech output can be added through text-to-speech synthesis using systems like ElevenLabs, Coqui TTS, or pyttsx3, creating a natural conversational audio experience.

A vision module can enable screenshot understanding through multimodal LLM APIs, document OCR and reading through libraries like Tesseract or AWS Textract, and camera-based visual assistance for users who require accessibility support. These vision capabilities would genuinely realize the 'multimodal' designation in AAROH's name.

14.2 Enhanced Intelligence

On the intelligence side, the keyword-driven classifier can evolve into a learned intent ranking model trained on domain-specific interaction data. A hybrid LLM-based classifier with confidence thresholds could handle ambiguous or indirect commands more gracefully than pure regex matching. The addition of chain-of-thought reasoning to the task planning pipeline could enable multi-step task orchestration, where AAROH breaks complex user goals into sequences of atomic actions.

14.3 Advanced Safety and Compliance

The safety layer can be extended with multi-step approval flows that require explicit user confirmation before executing high-impact actions. Audit trails that log all executed actions with timestamps and user identifiers would support compliance requirements in enterprise deployments. Sandboxed tool execution using containerization technologies would provide hardware-level isolation for potentially risky operations.

14.4 Richer Personalization

The user profile component can be expanded to retain long-term preferences, work habits, frequently used applications, allowed integration endpoints, and personalized assistant

behavior patterns. This richer personalization would enable AAROH to proactively suggest actions based on historical patterns and adapt its communication style to the individual user's preferences over time.

14.5 Systems Integration and Deployment

From a systems perspective, future versions could support plug-in adapters for calendar systems such as Google Calendar and Outlook, messaging platforms like Slack and Teams, file search and management, browser automation through frameworks like Playwright or Selenium, note-taking applications, and institution-specific internal services through secure API integrations.

Cross-device continuity represents another promising future direction, in which the same user context and preferences could be accessed across laptop, mobile, and cloud sessions while preserving strong privacy guarantees through end-to-end encryption of the synchronization data. With these extensions, AAROH could evolve from a strong academic prototype into a practical, personalized AI operating layer for everyday computing.

CHAPTER 15: CONCLUSION

This major project report presented the design and implementation of AAROH, a Multimodal, Context-Aware and Action-Centric Artificial Intelligence System developed as a solo B.Tech project. The work addressed an important limitation of modern AI assistants: the divide between conversational ability and real-world action. By integrating a FastAPI backend, a Streamlit frontend, a hybrid classifier, structured intent extraction, a permission-aware safety guard, short-term memory, semantic memory, and bounded action adapters, the project demonstrates that these capabilities can be combined into a coherent and extensible architecture.

The report showed that AAROH is not merely a chatbot and not merely an automation script. It is a hybrid system in which intelligence, memory, and action are organized through modular engineering principles. Functional evaluation based on source-code-backed scenarios supports the viability of the design. At the same time, the report honestly acknowledges that the current implementation remains a prototype with expansion opportunities in multimodal processing, broader tool integration, richer personalization, and stronger deployment maturity.

In summary, AAROH stands as a meaningful final-year contribution in the field of intelligent systems. It reflects current directions in AI toward tool use, contextual reasoning, retrieval-augmented generation, and human-centered automation. The project offers both an implemented software artifact and a forward-looking architectural framework for future research and development.

CHAPTER 16: REFERENCES

The following references are listed in order of citation throughout this report, in compliance with the IEEE citation format as specified in the project report guidelines.

- [1] G. Vamshi, "AAROH: A Multimodal, Context-Aware and Action-Centric Artificial Intelligence System," GitHub repository, 2026. [Online]. Available: <https://github.com/GajulaVamshi/AAROH>
- [2] FastAPI, "FastAPI Documentation," [Online]. Available: <https://fastapi.tiangolo.com/>
- [3] Streamlit, "Streamlit Documentation," [Online]. Available: <https://docs.streamlit.io/>
- [4] LangChain, "LangChain Python Documentation," [Online]. Available: <https://python.langchain.com/>
- [5] Chroma, "Chroma Documentation," [Online]. Available: <https://docs.trychroma.com/>
- [6] Groq, "Groq Developer Documentation," [Online]. Available: <https://console.groq.com/docs>
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Proc. NeurIPS, 2017.
- [8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language Models are Few-Shot Learners," in Proc. NeurIPS, 2020.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-T. Yih, T. Rocktaschel et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019.
- [11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019.
- [12] Meta AI, "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.

- [13] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.
- [15] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in Proc. NeurIPS, 2023.
- [16] Y. Bai, S. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan et al., "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback," arXiv preprint arXiv:2204.05862, 2022.
- [17] Sentence Transformers, "all-MiniLM-L6-v2 Model Card," [Online]. Available: <https://www.sbert.net/>
- [18] Harrison Chase, "LangChain," GitHub repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>

CHAPTER 17: APPENDICES

Appendix A: Key API Schema Summary

Schema	Field	Meaning
AAROHRequest	query	User input text
AAROHRequest	user_id	User/session identifier
AAROHResponse	response	Assistant output text
AAROHResponse	intent_type	Derived intent classification
AAROHResponse	status	Execution status
AAROHResponse	timestamp	Response creation time

Table A.1: Request and Response Schema Summary

Appendix B: Installation Notes

1. Install Python 3.10 or later.
2. Install project dependencies from the requirements file: `pip install -r requirements.txt`
3. Create a `.env` file with optional `GROQ_API_KEY`, `EMAIL_USER`, and `EMAIL_PASS` values.
4. Run the FastAPI backend: `python app.py`
5. Run the Streamlit frontend in a separate terminal: `streamlit run frontend.py`
6. Ensure browser access is available for web and media demo actions.

Appendix C: Abbreviated Source Tree

The project source tree is organized as follows:

```
AAROH/  
├── actions/  
│   ├── messaging.py  
│   ├── system_actions.py  
│   └── web_actions.py  
├── api/  
├── context/  
│   ├── semantic.py  
│   ├── short_term.py  
│   └── user_profile.py  
├── core/  
│   ├── classifier.py  
│   ├── config.py  
│   ├── intent.py  
│   ├── llm.py  
│   ├── router.py  
│   └── safety.py  
└── engines/  
    ├── chat_engine.py  
    └── task_engine.py
```

```
| docs/  
| app.py  
| frontend.py  
| docker-compose.yml
```

End of Report

Gajula Vamshi (221020110015) | B.Tech CSE (Artificial Intelligence)

Om Sterling Global University, Hisar, Haryana, India | May 2026